required. If n is unspecified, then bins of integer width are used. Look at the following example:

```
$ gsl-randist 1246 5 gamma 1 2 >gamma.txt
$ gsl-histogram 0 2 5 <gamma.txt
0 0.4 1
0.4 0.8 0
0.8 1.2 2
1.2 1.6 1
1.6 2 0
```

In this example, we generate five random numbers from the Cauchy distribution, store them in a file, and use the file as input to the *gsl-histogram* command. Instead of using a file, a more dynamic way of doing this in one step would be to use a pipe, which would produce the same output as above:

```
$ gsl-randist 1246 5 gamma 1 2 | gsl-histogram 0 2 5
```

IEEE floating-point arithmetic

Most of us are aware that numbers are stored in binary form on a computer. The standards defined by IEEE are the most widely accepted and used by many hardware and software manufacturers. The most current version is IEEE 754-2008, released in August 2008 (source: Wikipedia). Let's take a look at some basic definitions:

- **Fixed-point arithmetic:** Imagine a simple example of currency. If I say that the total is Rs 10.51, it means that the total is ten rupees and fifty one paisa. Since there is no smaller monetary unit than a paisa, the number of digits after the decimal is restricted to two. Therefore, we truncate the number to two places after the decimal point; this is an example of fixed-point arithmetic.
- **Floating-point arithmetic:** On the contrary, look at pi or any scientific constant. You might recall that the speed of light is 299792458 meters per second, but most of us use 2.99 x 10^8 m/s to round it off. This is an example of floating-point arithmetic.
 Since in scientific computing, a lot of calculations are rounded off to some fixed number of digits, we actually always get an approximate answer. Let us take an in-depth look at IEEE 754 proposals:
- Finite numbers can be of base 2 (binary) or base 10 (decimal). The numerical value of a finite number is: *(−1) s × c × bq*, where *s* is a sign (zero or one), *c* is a significand (or 'coefficient'), *q* is an exponent, and b is the base (2 or 10). For example, if the sign is 1 (indicating negative), the significand is 22222, the exponent is −4, and the base is 10, then the value of the number is −2.2222.
- Infinities at both extremes: +∞ and −∞.

The format for single-precision numbers uses 32 bits, divided into *sign bit* - 1 bit, *exponent* - 8 bits, and fraction or *significand* - 23 bits. The format for double-precision numbers uses 64 bits, divided into *sign bit* - 1 bit, *exponent* - 11 bits, and fraction or *significand* - 52 bits.

It could be quite helpful and exciting to check out variables at bit-level and, surprisingly, GSL has routines that can help you do this. Let's look at an example that might help you understand this better. Using IEEE library functions is similar to using any other GSL function. All the IEEE function codes are defined in the header file *gsl_ieee_utils.h*. You can take a quick look at it to understand the working of the code. There are some functions available, and you can set an environment variable before you run the program, which will be used by the GSL IEEE code to determine the mode to use.

The following program first prints a number in readable format; then it prints it in binary format, using GSL IEEE functions. The idea is to show how values can vary according to the precision level, rounding off, etc. Take a closer look at the values, and try to catch the differences.

```
#include <stdio.h>
#include <stdlib.h>
#include <gsl/gsl_ieee_utils.h>
int main (void)
{
      double f =10.0;
      float fd = f;
      gsl_ieee_env_setup (); /* read GSL_IEEE_MODE from command line*/
      f= 100 * (double) (1.0 * (rand() / (RAND_MAX + 0.0)));
      fd= -100 * (1.0 * (rand() / (RAND_MAX + 0.0)));
      printf (" f=%.10f ->",f); gsl_ieee_printf_double(&f);
      printf ("\n");
      printf ("fd=%.12lf ->",fd); gsl_ieee_printf_float(&fd);
      printf ("\n");
      return 0;
}
```

Compile the program with the following code:

```
$ gcc -Wall gsl_ieee.c -lgsl -lgslcblas -lm -o gsl_ieee
```

We invoke it with different IEEE modes:

```
$./gsl_ieee
 f=84.8187717155 -> 1.0101000000010011001111000111001001000000000100110011111^2^6
fd=-39.438293457031 ->-1.0011101110000001101000^2^5
$ GSL_IEEE_MODE="round-to-nearest" ./gsl_ieee
GSL_IEEE_MODE="round-to-nearest,trap-common"
 f=84.8187717155 -> 1.0101000000010011001111000111001001000000000100110011111^2^6
fd=-39.438293457031 ->-1.0011101110000001101000^2^5
$ GSL_IEEE_MODE="single-precision" ./gsl_ieee
GSL_IEEE_MODE="single-precision,trap-common"
 f=84.8187750399 -> 1.0101000000010011001111010000000000000000000000000000^2^6
fd=-39.438293457031 ->-1.0011101110000001101000^2^5
$ GSL_IEEE_MODE="double-precision" ./gsl_ieee
GSL_IEEE_MODE="double-precision,trap-common"
 f=84.8187717155 -> 1.0101000000010011001111000111001001000000000100110011111^2^6
```